

untangle::actuator

Generated by Doxygen 1.18.0

1 actuator	1
2 Topic Index	5
2.1 Topics	5
3 Namespace Index	7
3.1 Namespace List	7
4 Concept Index	9
4.1 Concepts	9
5 Class Index	11
5.1 Class List	11
6 File Index	13
6.1 File List	13
7 Topic Documentation	15
7.1 namespace untangle: functions	15
7.1.1 Detailed Description	15
7.1.2 Function Documentation	15
7.1.2.1 connect() [1/4]	15
7.1.2.2 connect() [2/4]	17
7.1.2.3 connect() [3/4]	17
7.1.2.4 connect() [4/4]	18
7.1.2.5 bind() [1/2]	19
7.1.2.6 bind() [2/2]	19
8 Namespace Documentation	21
8.1 untangle Namespace Reference	21
8.1.1 Detailed Description	22
8.1.2 Typedef Documentation	22
8.1.2.1 task	22
8.1.3 Function Documentation	23
8.1.3.1 last_arg()	23
8.1.3.2 bind_task()	23
8.1.3.3 connect_one()	24
8.1.3.4 connect_named_one()	25
9 Concept Documentation	27
9.1 untangle::task_callback_for Concept Reference	27
9.1.1 Concept definition	27
9.1.2 Detailed Description	27
10 Class Documentation	29
10.1 untangle::actuator< action_t > Struct Template Reference	29

10.1.1 Detailed Description	31
10.1.2 Member Typedef Documentation	32
10.1.2.1 actions_t	32
10.1.2.2 tasks_t	32
10.1.2.3 results_t	32
10.1.2.4 errors_t	33
10.1.3 Constructor & Destructor Documentation	33
10.1.3.1 actuator()	33
10.1.4 Member Function Documentation	33
10.1.4.1 operator=()	33
10.1.4.2 type()	34
10.1.4.3 reset()	34
10.1.4.4 copy_from()	34
10.1.4.5 release_owned()	35
10.1.4.6 invoke_callback()	35
10.1.4.7 operator()()	36
10.1.4.8 invoke_action()	37
10.1.4.9 call_tasks()	37
10.1.4.10 add() [1/4]	38
10.1.4.11 add() [2/4]	39
10.1.4.12 add() [3/4]	39
10.1.4.13 add() [4/4]	39
10.1.4.14 add_task()	40
10.1.4.15 remove() [1/2]	41
10.1.4.16 remove() [2/2]	41
10.1.4.17 is_connected()	42
10.1.4.18 has_tasks()	42
10.1.4.19 has_action()	42
10.1.5 Member Data Documentation	43
10.1.5.1 tasks	43
10.1.5.2 owned	43
10.2 untangle::invalid_action Struct Reference	43
10.2.1 Detailed Description	44
10.2.2 Constructor & Destructor Documentation	44
10.2.2.1 invalid_action()	44
10.2.3 Member Function Documentation	44
10.2.3.1 what()	44
11 File Documentation	45
11.1 actuator.hpp	45
Index	51

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

Anonymous lambdas as actions

An actuator normally does not own its actions: it stores pointers to `std::function` objects the caller keeps alive. `add()` also has an overload that takes an action by value, so an anonymous lambda can be an action with no named variable to keep around, and returns a handle to pass to `remove()`:

```
untangle::actuator<std::function<int(int)>> actuator_scale;
auto* handle = actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add("triple", [](int v) { return v * 3; });
```

`connect()` accepts them too, but it deduces the action type from its arguments, and a lambda has its own closure type – it is not a `std::function` until something converts it. A call made only of anonymous lambdas therefore has nothing to deduce from, and the signature has to be named on `connect()` itself:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { /* ... */ }, [](int angle) { /* ... */ });
```

Naming the type of the variable the result is assigned to does not supply it – template arguments are deduced from the call arguments alone, never from what the returned value is assigned to:

```
// ill formed: "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { /* ... */ });
```

One named action anywhere in the call deduces the type for the whole of it, and the anonymous lambdas beside it then need nothing:

```
auto actuator_rotate = untangle::connect(action1, [](int angle) { /* ... */ });
```

The named form of `connect()` works the same way: a pair holding a pointer names an action the caller owns, a pair holding the action by value hands it over to the actuator, and the two can be mixed. The action type is deduced from the pointer, so a call whose first pair holds a lambda has to name the signature:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { /* ... */ }),
    std::make_pair("circle", [](int angle) { /* ... */ }));

// deduced from the leading pointer, so the lambda beside it needs nothing
auto actuator_mixed = untangle::connect(std::make_pair("triangle", &action1),
    std::make_pair("circle", [](int angle) { /* ... */ }));
```

Tasks: an action bound to its arguments and to the callback it must notify

An action is a subscription: it lives across invocations, and it is fired with a pack the caller supplies each time. A task is a one-shot. It carries its own arguments, is fired once, and reports what it produced to a callback it was built with. The two live side by side in one actuator and share nothing else – `operator()` fires the actions and leaves the tasks alone, `call_tasks()` fires the tasks and leaves the actions alone.

A task exists so that work which runs later than it was described can still say what became of it. That is what a queue needs, and an actuator holding only actions cannot give: the callback convention on `operator()` reads the trailing argument of the invocation, and a queued call has no invocation left to read from.

```
untangle::actuator<std::function<int(int)>> actuator_scale;

// the callback comes last, after the arguments the action is bound to
actuator_scale.add_task(untangle::bind_task(action, 21, [](int result) { /* result == 42 */ }));

actuator_scale.call_tasks(); // runs it, notifies it, and empties the list
```

A task without a callback does not exist. That is the whole difference from an action's callback, which is optional and is recognised by its type – a trailing argument that does not look like a callback is simply an argument, and nothing fires. A task's callback is the last argument of `bind_task()`, taken by position, and `task_callback_for` only checks that it can serve. There is no answer "no" to fall through to, so a missing or unusable callback is a `static_assert` rather than silence.

It is not forwarded to the action either, so an action needs no callback parameter of its own – and a task whose action returns void is a task like any other. It reports with a void() callback, because finished is the message and the result is optional:

```
untangle::actuator<std::function<void(int)>> actuator_notify;
actuator_notify.add_task(untangle::bind_task(action, 5, [] { /* finished */ }));
```

The arguments are copied when the task is built, while the caller still holds them, and handed to the action as the task's own lvalues when it runs. A task outlives the call that described it and cannot borrow what may already be gone; a move-only argument therefore does not compile.

add_task() answers bool. It refuses a task that has nothing to run or no callback to notify, and the refusal is the whole report – nothing is thrown and nothing is stored. Both are the caller's own mistake, and both are caught while the caller is still on the stack rather than surfacing later from call_tasks() as a std::bad_function_call recorded against a task whose action had in fact succeeded.

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to actuator::errors, the tasks behind it still run, and a caller relying on the callback alone will never hear about it. A callback runs inside the same try as the task that owns it, so what it throws travels the same path – errors can hold a failure for a task that succeeded.

A task's result goes to its callback and nowhere else: actuator::results is how an action hands back what it returned, and a task was built with something better. call_tasks() appends to errors rather than clearing it, while operator() clears both as it starts – so an actuator fired as one(); one.call_tasks(); reports both kinds together, and the actions go first.

Ask has_tasks() whether a task is waiting. is_connected() answers for the actions and for nothing else: an actuator holding only tasks is not connected, and a queue reading the wrong one would report a batch of tasks as nothing to do.

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in doc/refman.pdf for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [15](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::actuator functor	21
--------------------------	---	--------------------

Chapter 4

Concept Index

4.1 Concepts

Here is a list of all documented concepts with brief descriptions:

[untangle::task_callback_for](#)

Can `callback_t` be called as the completion callback of a task returning `result_t`? . . . [27](#)

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::actuator< action_t >	
An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>)	
	29
untangle::invalid_action	
Invalid action exception	43

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

actuator.hpp	45
------------------------------	----

Chapter 7

Topic Documentation

7.1 namespace untangle: functions

Functions

- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (action_t &A1, Actions &&... An)`
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
requires (!std::is_same_v<value_t, action_t*>)
`actuator< action_t > untangle::connect (std::pair< key_t, value_t > A1, Actions &&... An)`
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>`
`action_t untangle::bind (class_t *obj, T class_t::*method)`
Binding to a class method.

7.1.1 Detailed Description

7.1.2 Function Documentation

7.1.2.1 connect() [1/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &&... An)
```

Creates an actuator holding an initial list of actions.

Remarks

Ownership: an action passed as an lvalue is one the caller owns, and the actuator only points at it – it must outlive the actuator. An action passed as an rvalue, an anonymous lambda above all, is moved into the actuator instead and needs no named variable at all. The two can be mixed in one call. See the ownership convention on [actuator](#).

Warning

A call made only of anonymous lambdas can not deduce `action_t`. A lambda has its own closure type and is not a `std::function` until something converts it, so there is nothing in such a call to deduce the action signature from, and it has to be named:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { ... }, [](int angle) { ... });
```

Warning

Naming the type of the variable the result is assigned to does not supply it:

```
// still ill formed -- "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { ... });
```

Template arguments are deduced from the call arguments alone, never from what the returned value is assigned to, so the explicit argument belongs on [connect\(\)](#) itself. One named action anywhere in the call deduces `action_t` for the whole of it, and the anonymous lambdas beside it then need nothing.

Parameters

A1	- The first action. It is specified as <code>std::function<...></code> .
An	- Any number of further actions, each convertible to the type of A1.

Returns

An [actuator](#).

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
    for (const auto& s : shapes) {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);

// connect() deduces the action type from its arguments. A lambda has its own closure type
// and is not a std::function until something converts it, so a call made only of anonymous
// lambdas has nothing to deduce from and the signature has to be named on connect()
// itself. Naming the type of the variable assigned to would not supply it: template
// arguments are never deduced from what the returned value is assigned to.
auto actuator_scale = untangle::connect<std::function<int(int)>>([](int v) { return v * 2; },
    [](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

7.1.2.2 connect() [2/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::type_identity_t< action_t > && A1,
    Actions &&... An)
```

Creates an actuator owning an initial list of actions.

This overload takes the first action by rvalue, so it is the one that accepts a leading anonymous lambda. `std::type_identity_t` makes `action_t` a non-deduced context here, which is what both keeps this overload out of the way of the lvalue one – it is not a candidate at all unless `action_t` is named – and states the requirement described by the warning on the overload taking the first action by lvalue.

Parameters

A1	- The first action, as an rvalue.
An	- Any number of further actions, owned or pointed at by value category.

Returns

An **actuator**.

7.1.2.3 connect() [3/4]

```
template<typename action_t, typename key_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::pair< key_t, action_t * > A1,
    Actions &&... An)
```

Creates an actuator holding an initial map of named actions.

Remarks

Ownership works as it does for the unnamed **connect()** overloads: a pair holding a pointer names an action the caller owns, and a pair holding an action by value – an anonymous lambda above all – hands it to the actuator to own. The two can be mixed in one call.

Warning

The action type is deduced from the pointer in the first pair, so a call whose first pair holds an anonymous lambda has nothing to deduce it from and has to name the signature on **connect()** itself, exactly as the unnamed overload does:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { ... })),
    std::make_pair("circle", [](int angle) { ... }));
```

Remarks

`action_t` leads the template parameter list, ahead of `key_t`, so that naming it – **connect**<std::function<...>>(...) – means the same thing on every **connect()** overload. Both parameters are deduced from the first pair whenever it holds a pointer, so nothing has to be named here in the first place.

Parameters

A1	- The first name/action pair. Its second element is a pointer to an action.
An	- Any number of further name/action pairs, owned or pointed at according to whether they hold the action itself or a pointer to it.

Returns

An [actuator](#).

Example:

```
// A pair holding the action itself hands it to the actuator to own. The action type is
// deduced from a pointer, and there is no pointer here, so the signature has to be named
// on connect() -- as it does for the unnamed overload.
auto actuator_scale = untangle::connect<std::function<int(int)>>(
    std::make_pair("double", [](int v) { return v * 2; }),
    std::make_pair("triple", [](int v) { return v * 3; }));

actuator_scale.invoke_action("double", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20));
actuator_scale.invoke_action("triple", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(30));
```

7.1.2.4 connect() [4/4]

```
template<typename action_t, typename key_t, typename value_t, typename... Actions>
requires (!std::is_same_v<value_t, action_t*>)
actuator< action_t > untangle::connect (
    std::pair< key_t, value_t > A1,
    Actions &&... An)
```

Creates an actuator owning an initial map of named actions.

This overload takes a first pair holding the action itself rather than a pointer to it, so it is the one that accepts a leading anonymous lambda. `action_t` is never deduced here – the `requires` clause is what keeps it from competing with the overload taking a pointer in the first pair over a leading pointer pair, which both would otherwise accept once `action_t` is named.

Parameters

A1	- The first name/action pair, holding the action itself.
An	- Any number of further name/action pairs.

Returns

An [actuator](#).

7.1.2.5 bind() [1/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>>
action__t untangle::bind (
    const std::shared_ptr< class__t > & obj,
    T class__t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action__t - A `std::function` that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid_action](#).

7.1.2.6 bind() [2/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>>
action__t untangle::bind (
    class__t * obj,
    T class__t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is detected and reported as a dead action, but a pointer to an already destroyed object can not be distinguished from a valid one. Prefer the overload taking a `std::shared_ptr`.

Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Binding a null pointer produces an action that is always dead: invoking it throws [invalid_action](#).

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action_t - A `std::function` that wraps the pointer to function member.

Chapter 8

Namespace Documentation

8.1 untangle Namespace Reference

Interface to `untangle::actuator` functor.

Classes

- struct `invalid_action`
Invalid action exception.
- struct `actuator`
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Concepts

- concept `task_callback_for`
Can `callback_t` be called as the completion callback of a task returning `result_t`?

Typedefs

- using `task` = `std::function<void()>`
An action bound to its arguments, and to the callback it notifies once it has finished – all three sealed into one nullary callable.

Functions

- template<typename... Args>
auto `last_arg` (Args &&... args)
Get last argument of params pack;.
- template<typename action_t, typename... Args>
`task bind_task` (action_t action, Args &&... args)
Binds an action to its arguments and to the callback it must notify, which comes last.
- template<typename action_t, typename arg_t>
void `connect_one` (`actuator`< action_t > &target, arg_t &&arg)
Add one `connect()` argument to the actuator being built.

- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (action_t &A1, Actions &&... An)`
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename entry_t>`
`void connect_named_one (actuator< action_t > &target, entry_t &&entry)`
Add one named `connect()` argument to the actuator being built.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
requires (!std::is_same_v<value_t, action_t*>)
`actuator< action_t > connect (std::pair< key_t, value_t > A1, Actions &&... An)`
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵`
`T>::type>>`
`action_t bind (class_t *obj, T class_t::*method)`
Binding to a class method.

8.1.1 Detailed Description

Interface to `untangle::actuator` functor.

8.1.2 Typedef Documentation

8.1.2.1 task

using `untangle::task = std::function<void()>`

An action bound to its arguments, and to the callback it notifies once it has finished – all three sealed into one nullary callable.

What `actuator::add_task()` holds and `actuator::call_tasks()` fires. One is built by `bind_task()`, which binds the arguments and takes the callback off the end of the same pack.

Remarks

An action is a subscription; a task is a one-shot. An action lives across invocations and is fired with a pack the caller supplies each time. A task carries its own arguments, is fired once, and reports to its own callback. That is why a task is held by value rather than by pointer, and why nothing removes one – `call_tasks()` consumes it.

Running a task is what reports. It returns nothing because its result goes to its callback and nowhere else, so there is no second call to make and no part of it to reach from outside: a caller holding one can only fire it.

Neither the argument types nor the result type appear here, which is what lets a queue of tasks exist at all: one container holds tasks built from completely different actions, whatever they take and whatever they return.

Anything that can be a `std::function<void()>` is a task, so a caller with work already bound needs no `bind_task()` to hand it over. What makes it a task is how an actuator treats it: fired once, by `actuator::call_tasks()`, and consumed.

8.1.3 Function Documentation

8.1.3.1 `last_arg()`

```
template<typename... Args>
auto untangle::last_arg (
    Args &&... args)
```

Get last argument of params pack;.

Template Parameters

Args	params template types
------	-----------------------

Parameters

args	params
------	--------

Returns

last param value in case it exist, otherwise 0.

8.1.3.2 `bind_task()`

```
template<typename action_t, typename... Args>
task untangle::bind_task (
    action_t action,
    Args &&... args)
```

Binds an action to its arguments and to the callback it must notify, which comes last.

What builds a [task](#). The arguments and the callback – which is the last element of the same pack – are sealed into the action, and running what comes back invokes the action and then hands the callback what it returned. A task built here runs later than it was built – that is what a queue of them is for – so the arguments are copied now, while the caller still holds them, and handed to the action as the task's own lvalues when it runs.

Remarks

The callback is the last argument, by position. It cannot be a parameter of its own: a parameter pack cannot be followed by another parameter and still be deduced, so it arrives inside args and is split off here. That split is paid once – every caller above forwards (action, args...) and never learns of it.

It is not a trailing argument recognised by its type, which is what an action's callback convention does; see the convention on [actuator](#). Here the last argument is the callback, and [task_callback_for](#) only checks that it can serve as one. There is no answer "no" to fall through to, which is what lets a task's callback be required.

It is not forwarded to the action either. The action is invoked with args minus its last element, so an action needs no callback parameter of its own – and a void action, which could never have used one, is a task like any other.

Attention

Each argument is copied, so a move-only one does not compile. The task outlives the call that built it and cannot borrow what the caller may already have destroyed.

A task that could not do its job is built empty, and `actuator::add_task()` refuses an empty task. There are two such cases and both are the caller's own mistake: an empty callback, which the signature cannot rule out because an empty `std::function` satisfies `task_callback_for`, and an empty action, which has nothing to run. Caught here rather than surfacing from `actuator::call_tasks()` as a `std::bad_function_call`, where it would be read as the failure of a task that had in fact run. Only a callable that can be tested is tested: a bare lambda is never empty and is never refused for failing a test it cannot take.

Template Parameters

<code>action_↵ _t</code>	Type of the action. Any callable naming a <code>result_type</code> will do; it need not be a <code>std::↵ function</code> .
<code>Args</code>	Types of the arguments to bind, of which the last is the callback.

Parameters

<code>action</code>	- The action to bind.
<code>args</code>	- The arguments to bind to action, followed by the callback to notify.

Returns

The task, ready to be held by `actuator::add_task()` and fired by `actuator::call_tasks()` – or an empty one, which `add_task()` refuses. See the attention above.

8.1.3.3 `connect_one()`

```
template<typename action_t, typename arg_t>
void untangle::connect_one (
    actuator< action_t > & target,
    arg_t && arg)
```

Add one `connect()` argument to the actuator being built.

An argument that is an lvalue of the action type is one the caller owns: only its address is stored, and it has to outlive the actuator. Anything else – an anonymous lambda, a temporary `std::function`, a named lambda that is not an `action_t` yet – has no owner to outlive the actuator, so it is converted to an action and moved into it.

Remarks

Empty actions are dropped rather than stored, which is what `connect()` has always done with them. Dropping an empty owned action before it is stored is what keeps `actuator::owned` free of entries nothing points at.

Template Parameters

action↔ _t	Action type of the actuator being built.
arg_t	Deduced type of the argument, carrying its value category.

Parameters

target	- The actuator being built.
arg	- One connect() argument.

8.1.3.4 connect_named_one()

```
template<typename action_t, typename entry_t>
void untangle::connect_named_one (
    actuator< action_t > & target,
    entry_t && entry)
```

Add one named [connect\(\)](#) argument to the actuator being built.

It is [connect_one\(\)](#) for the named overloads: the second element of the pair decides ownership. A pointer to an action is one the caller owns and is only stored; anything else – an anonymous lambda, a temporary `std::function` – is converted to an action and moved into the actuator.

Remarks

A null pointer is dropped rather than stored, which is what the named [connect\(\)](#) has always done with one. An owned action that is empty is dropped for the same reason the unnamed [connect_one\(\)](#) drops one: nothing would ever point at it.

Template Parameters

action↔ _t	Action type of the actuator being built.
entry_t	Deduced type of the pair, carrying its value category.

Parameters

target	- The actuator being built.
entry	- One name/action pair.

Chapter 9

Concept Documentation

9.1 `untangle::task_callback_for` Concept Reference

Can `callback_t` be called as the completion callback of a task returning `result_t`?

```
#include <actuator.hpp>
```

9.1.1 Concept definition

```
template<typename callback_t, typename result_t>
concept task_callback_for =
    requires(callback_t& c, result_t& r) { requires std::is_void_v<decltype(c(r))>; } ||
    (std::is_void_v<result_t> &&
     requires(callback_t& c) { requires std::is_void_v<decltype(c())>; })
```

9.1.2 Detailed Description

Can `callback_t` be called as the completion callback of a task returning `result_t`?

A task's callback is required, and it is the last argument given to `bind_task()` rather than a trailing argument recognised by its type. So the question this asks is not whether the argument is a callback – it is one, by position – but only whether it can serve as one.

Remarks

That is the whole difference from an action's callback convention, which asks the whether and is allowed to answer no: a trailing argument that is not a callback is simply an argument, and nothing is invoked. A task has no such answer to fall through to, which is what lets the callback be required. See the callback convention on the actuator itself, and the warning that comes with it.

A task returning nothing still reports that it finished, so for `result_t` of `void` the callback takes no argument at all. The disjunct that asks to be handed a result yields false there rather than failing hard – forming `void&` in a `requires`-parameter list is a substitution failure in the immediate context – which leaves the `void` disjunct to decide. Its `std::is_void_v` guard is not decoration either: without it a `std::function<void()>` would satisfy this for any result type, and a task's result would go unreported.

Template Parameters

<code>callback_</code> <code>t</code>	Type of the callback.
<code>result_</code> <code>t</code>	Result type of the task it reports on.

Chapter 10

Class Documentation

10.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

Public Types

- using `actions_t` = `std::list<action_t*>`
Actions container type.
- using `tasks_t` = `std::list<task>`
Tasks container type.
- using `results_t` = `std::vector<typename result_t::type>`
Results container type.
- using `errors_t` = `std::vector<std::exception_ptr>`
Errors container type.

Public Member Functions

- `actuator` (const actuator &other)
Copy constructor.
- actuator & `operator=` (const actuator &other)
Assignment operator.
- action_t `type` () const
Helper method to access the action type of this object.
- void `reset` ()
Remove all actions, and any stored results and errors.
- void `copy_from` (const actuator &other)
Copy another actuator, re-pointing the copied actions at this object's storage.
- void `release_owned` (const action_t *action)
Destroy an owned action that nothing points at any more.
- template<typename last_t>
void `invoke_callback` (last_t &last)
Invoke the trailing callback argument, if the invocation has one.

- `template<typename... Args>`
`void operator()` (`Args &&... args`)
The call operator.
- `template<typename... Args>`
`void invoke_action` (`const std::string &name, Args &&... args`)
Invokes one single action associated with a key.
- `void call_tasks` ()
Fires every task held, notifies each callback, and empties the list.
- `void add` (`action_t *action`)
Add an action to the actions list.
- `action_t * add` (`action_t &&action`)
Add an action the actuator owns.
- `void add` (`const std::string &name, action_t *action`)
Add action to the actions map associated with a name.
- `action_t * add` (`const std::string &name, action_t &&action`)
Add an action the actuator owns, associated with a name.
- `bool add_task` (`task &&one`)
Adds a task, to be fired by `call_tasks()`.
- `void remove` (`const action_t *action`)
Remove an action from the actions list.
- `void remove` (`const std::string &name`)
Remove an action from actions map.
- `bool is_connected` () `const`
Check if this actuator is "connected" with other actions.
- `bool has_tasks` () `const`
Is there a task waiting to be fired?
- `bool has_action` (`const std::string &name`) `const`
Check if there is certain named action.

Public Attributes

- `actions_t` `actions`
Actions list.
- `actions_map_t` `actions_map`
Named actions map.
- `results_t` `results`
Actions return values list.
- `errors_t` `errors`
What the actions threw. See `errors_t`.
- `tasks_t` `tasks`
Tasks waiting to be fired, in the order `add_task()` took them.
- `std::list< action_t >` `owned`
Actions this actuator owns, as added by `add(action_t&&)`.

10.1.1 Detailed Description

```
template<typename action_t>
struct untangle::actuator< action_t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Remarks

An actuator object can be constructed with an initial list of actions by [connect\(\)](#).

Callback convention: when the last argument of an invocation can be called with the action return type `R` and returns nothing, it is not passed on as a plain argument alone – it is also treated as a completion callback. The action is invoked with the full argument list as usual, and the callback is then invoked with the action return value. Any void returning callable qualifies: a `std::function<void(R)>`, a lambda taking `R`, a function pointer. It applies to [operator\(\)](#) and to [invoke_action\(\)](#) alike. Actions returning void have no result to report, so no callback is invoked for them.

Warning

The return type is what separates a callback from an argument the action means to consume itself. A trailing callable that returns a value – a transform, a comparator – is left alone, and by the same rule a callback written to return something is silently not invoked. A callback returns nothing.

Remarks

Argument convention: an action must not take ownership of the arguments it is invoked with. [operator\(\)](#) forwards one argument pack to every action in the list, and an rvalue can be moved from only once, so an action that consumes an argument leaves the actions after it holding a moved-from object – an empty `std::function` for a callback a later action means to call itself. The arguments are forwarded rather than copied so that a single action pays nothing for the broadcast; what that buys has to be respected by the actions.

Warning

An action does not have to move an argument in its body to consume it: a parameter taken by value is move constructed from an rvalue by the call itself. Take arguments by reference or const reference, or invoke with lvalues, whenever more than one action is connected. The behaviour is otherwise unpredictable, and it is the caller and the action signatures together that decide it – an actuator cannot detect the violation.

Remarks

[invoke_action\(\)](#) invokes one single action, so nothing follows it and the convention does not constrain it.

Failure convention: an action that throws does not stop the invocation. What it threw is stored in [actuator::errors](#), one `std::exception_ptr` per failure, and the actions after it still run. An [untangle::invalid_action](#) means a dead binding, so that action is dropped as well; anything else leaves the action in place. Nothing is printed and nothing escapes the call - the caller reads [actuator::errors](#) and decides.

Ownership convention: an actuator normally does not own its actions. It stores pointers to `std::function` objects the caller keeps alive, and those have to outlive it. [add\(action_t&&\)](#) is the exception – it moves the action into [actuator::owned](#) and points at the stored copy – which is what lets [add_anonymous_lambda](#) be an action, with no named variable to keep around. The handle it returns is the only way to [remove\(\)](#) such an action afterwards.

Template Parameters

action↔ _t	Action type. It is specified as <code>std::function<...></code> .
---------------	---

10.1.2 Member Typedef Documentation

10.1.2.1 actions_t

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the [remove\(\)](#) operation. `std::function` supports only equality operator for `nullptr` (two `std::function(s)` can not compare).

10.1.2.2 tasks_t

```
template<typename action_t>
using untangle::actuator< action_t >::tasks_t = std::list<task>
```

Tasks container type.

Remarks

The elements are values, not pointers. An action is stored by pointer because [remove\(\)](#) needs identity and two `std::function(s)` cannot be compared; a task is never removed one at a time – [call_tasks\(\)](#) consumes the whole list – so it needs neither.

A `std::list` for the same reason [owned](#) is one: adding never invalidates the address of an element already in it, so a reference taken into the list stays good.

It names no result type, because [untangle::task](#) does not: what this actuator holds tasks of has nothing to do with the actions it fires.

10.1.2.3 results_t

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

10.1.2.4 errors_t

```
template<typename action_t>
using untangle::actuator< action_t >::errors_t = std::vector<std::exception_ptr>
```

Errors container type.

It holds what the actions threw, as `std::exception_ptr`, in the order they were invoked.

10.1.3 Constructor & Destructor Documentation

10.1.3.1 actuator()

```
template<typename action_t>
untangle::actuator< action_t >::actuator (
    const actuator< action_t > & other) [inline]
```

Copy constructor.

Remarks

It can not be defaulted, see [copy_from\(\)](#).

10.1.4 Member Function Documentation

10.1.4.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [inline]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by decltype(<[actuator](#) instance>.[type\(\)](#))

Returns

action_t

10.1.4.3 reset()

```
template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]
```

Remove all actions, and any stored results and errors.

After this call the actuator is empty: [actuator::is_connected\(\)](#) returns false. The actions it owns are destroyed, so every handle returned by [add\(action_t&&\)](#) is dangling afterwards; the actions it merely points at are left untouched.

10.1.4.4 copy_from()

```
template<typename action_t>
void untangle::actuator< action_t >::copy_from (
    const actuator< action_t > & other) [inline]
```

Copy another actuator, re-pointing the copied actions at this object's storage.

Remarks

A defaulted copy is wrong as soon as an actuator owns anything: it duplicates [actuator::owned](#) but leaves [actuator::actions](#) pointing into other's copy of it, so the new object dangles the moment other is destroyed. The owned actions are copied first, then each pointer into other::owned is translated to the matching element here.

A pointer to an action the caller owns is external to both actuators and is copied unchanged: it is not in either [actuator::owned](#), so it is not in the translation table and is left as it is.

Parameters

other	- The actuator to copy. Self copy is the caller's to exclude.
-------	---

10.1.4.5 release_owned()

```
template<typename action_t>
void untangle::actuator< action_t >::release_owned (
    const action_t * action) [inline]
```

Destroy an owned action that nothing points at any more.

Remarks

Removing an action only drops a pointer to it. When the actuator owns that action the std::function itself lives in `actuator::owned` and has to go too, or a loop of `add(action_t&&)` and `remove()` grows `actuator::owned` without bound.

The action is kept while anything still refers to it: one handle can be added to both the list and the map, and removing it from one must not leave the other pointing at a destroyed object. An action the actuator does not own is not in `actuator::owned`, so nothing is found to erase and the caller's object is left alone.

Parameters

action	- The action just removed. A null pointer is accepted and matches nothing.
--------	--

10.1.4.6 invoke_callback()

```
template<typename action_t>
template<typename last_t>
void untangle::actuator< action_t >::invoke_callback (
    last_t & last) [inline]
```

Invoke the trailing callback argument, if the invocation has one.

Does nothing unless last_t can be called with the action return type and returns nothing, so any other trailing argument is left alone. The test is on what the type can do, not on what it is, so a plain lambda qualifies as well as a std::function. See the callback convention on `actuator`.

Remarks

The single condition does the work of two tests: (1) whether last can be called with the action return type at all, and (2) whether that call returns nothing. Spelled out as traits they need two nested if constexpr:

```
// (1)
if constexpr (std::is_invocable_v<last_t&, typename result_t::type>) {
    // (2)
    if constexpr (std::is_void_v<std::invoke_result_t<last_t&, typename result_t::type>)>) {
        last(results.back());
    }
}
```

Both are folded into the one expression instead, each in a different part of it:

```
requires { requires std::is_void_v<decltype( last(results.back()) )>; }
// \underline{(2)} \underline{(1)}
```

Remarks

(1) Invocability comes for free. To form `decltype(last(results.back()))` the compiler must first form the call `last(...)`. If `last` is an int, that expression does not exist – and inside a `requires-expression` a substitution failure yields `false` instead of being ill formed, the same property concepts are built on. So invocability is never tested separately: it is a precondition of the return type question, and failing it is what makes the whole condition false.

(2) The return type test is the inner `requires`, and the doubled keyword is not a typo. `requires { expr; }` asks only whether `expr` is well formed, and `std::is_void_v<...>` is well formed when it is `false` too – that spelling would let a value returning callable through. The second keyword makes it a nested requirement: a condition that has to hold, not merely compile.

The trait spelling has to nest, which is part of why it is not the one used here: the two `if constexpr` cannot be joined with `&&`. `&&` short circuits evaluation, not instantiation, so `std::invoke_result_t` is formed whatever the first trait says, and for a type that is not invocable at all it has no `::type` – a hard error rather than a false result.

The caller must copy the callback out of the argument pack before invoking the action: an action taking it by value moves from the caller's `std::function`, which would leave this an empty function to call.

Template Parameters

<code>last_↵ _t</code>	Type of the last argument, as returned by <code>last_arg()</code> .
----------------------------	---

Parameters

<code>last</code>	- Last argument value, copied before the action was invoked.
-------------------	--

10.1.4.7 `operator>()()`

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

The call operator.

Actions in the `actuator::actions` list are triggered by invoking the call operator.

Parameters

<code>args</code>	- Arguments list must match the action arity. A trailing callback is invoked once per action, with that action's return value; see the convention on <code>actuator</code> .
-------------------	--

Warning

The same argument pack is forwarded to every action in the list, so the argument convention on [actuator](#) applies here in full: an action that consumes an argument leaves the actions after it with a moved-from object.

Remarks

Nothing escapes this call: what an action throws goes to [actuator::errors](#), and the actions after it still run. See the failure convention on [actuator](#).

10.1.4.8 invoke_action()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

An action that can not be invoked – a null pointer, an empty std::function, a binding whose object is gone – is dropped from [actuator::actions_map](#) instead, exactly as [operator\(\)\(\)](#) drops one from [actuator::actions](#). Nothing is invoked and nothing escapes; [actuator::has_action\(\)](#) reports it gone afterwards.

Remarks

What the action throws is stored in [actuator::errors](#), as it is for [operator\(\)\(\)](#). See the failure convention on [actuator](#).

Parameters

name	- Key associated with the action. Invoking a key that is not in the map does nothing.
args	- Arguments list must match the action arity. A trailing callback is invoked with the action's return value; see the convention on actuator .

10.1.4.9 call_tasks()

```
template<typename action_t>
void untangle::actuator< action_t >::call_tasks () [inline]
```

Fires every task held, notifies each callback, and empties the list.

Running a task is what notifies it: the callback [bind_task\(\)](#) sealed into it is invoked with what the action returned – with nothing at all, for a task whose action returns void, because finished is the message and the result is optional. So this has no arm for a void result and none for a non-void one; the task knows which it is. Tasks run in the order [add_task\(\)](#) took them.

Remarks

It does not fire the actions, exactly as `operator>()()` does not fire the tasks. The two kinds share an actuator and nothing else.

A task's result is delivered to its callback and nowhere else. `actuator::results` is how an action hands back what it returned; a task was built with something better, and does not need both.

What anything throws is appended to `actuator::errors`, not written over it. `operator>()()` clears that list as it starts and this does not, so an actuator fired as `one(); one.call_tasks();` reports both kinds together – which means the actions go first. The other order loses what the tasks recorded.

Failure convention, as `operator>()()` has one: a task that throws does not stop the pass, and the tasks behind it still run.

Attention

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `actuator::errors` and nothing else is said. A caller relying on the callback alone will never hear about it – read the errors.

A callback runs inside the same try as the task that owns it, so what it throws travels the same path. `actuator::errors` can therefore hold a failure for a task whose action in fact succeeded. The task is consumed either way; re-running an action that already ran, to reach a callback that already threw, would be worse than losing the notification.

10.1.4.10 `add()` [1/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Add an action to the actions list.

Parameters

action	- Action to be added.
--------	-----------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.11 add() [2/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    action_t && action) [inline]
```

Add an action the actuator owns.

The action is moved into `actuator::owned`, so it needs no named variable outliving the actuator: an anonymous lambda can be passed straight in, and the call converts it to `action_t`. See the ownership convention on `actuator`.

Parameters

action	- Action to be added. It is taken by value and moved from.
--------	--

Returns

`action_t*` - A handle to the stored action, to pass to `remove()`. It stays valid until that action is removed or the actuator is destroyed. A copy of the actuator owns its own copy of the action and has its own handle to it.

Example:

```
// An anonymous lambda has no named variable to outlive the actuator, so the actuator takes
// ownership of it: it is moved into actuator::owned and the list points at the stored copy.
untangle::actuator<std::function<int(int)>> actuator_scale;
actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add([](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

10.1.4.12 add() [3/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    const std::string & name,
    action_t * action) [inline]
```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added.

10.1.4.13 add() [4/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    const std::string & name,
    action_t && action) [inline]
```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added. It is taken by value and moved from.

Returns

action_t* - A handle to the stored action, as `add(action_t&&)` returns, or nullptr when name is already taken. The name already in the map keeps the action it has, as it does for `add(const std::string&, action_t*)`, and the action offered here is dropped rather than left ownerless in `actuator::owned`.

10.1.4.14 add_task()

```
template<typename action_t>
bool untangle::actuator< action_t >::add_task (
    task && one) [inline]
```

Adds a task, to be fired by `call_tasks()`.

A task is built by `bind_task()`, which seals an action, its arguments and the callback it must notify into one nullary callable, and is stored here by value – the actuator owns every task it holds, unlike the actions it merely points at. Anything that can be an `untangle::task` is taken, so a caller with work already bound needs no `bind_task()`.

Remarks

There is no counterpart to `remove()`. `call_tasks()` consumes the list, so a task leaves by being fired; nothing needs to identify one afterwards.

Nor is there a named form. `actions_map` exists so `invoke_action()` can fire one action on demand; tasks are fired as a batch, and the order they were added in is the order they run in.

Attention

It refuses an empty task, and the refusal is the whole report – nothing is thrown and nothing is stored. A task notifies itself, so there is no callback here to test separately: `bind_task()` tests both halves and hands back an empty task when either the action or the callback could not do its job. Either way the refusal lands while the caller is still on the stack, rather than surfacing from `call_tasks()` as a `std::bad_function_call` recorded in `actuator::errors`. There it would be read as the failure of a task whose action had in fact succeeded, on whatever thread happened to fire it.

Parameters

one	- The task to add. It is taken by value and moved from; a refused one is dropped.
-----	---

Returns

true - the task was taken, and `call_tasks()` will fire it.

false - it was refused, because it is empty and so can neither run nor report. A task that cannot report is not a task; see the convention on `untangle::task`.

10.1.4.15 remove() [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const action_t * action) [inline]
```

Remove an action from the actions list.

An invalid action (empty std::function) is implicitly removed when `operator()()` is invoked.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See `release_owned()`.

Parameters

action	- Action to be removed. For an owned action this is the handle returned by <code>add(action_t&&)</code> .
--------	---

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

10.1.4.16 remove() [2/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const std::string & name) [inline]
```

Remove an action from actions map.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See `release_owned()`.

Parameters

name	- Name of the action to remove.
------	---------------------------------

10.1.4.17 `is_connected()`

```
template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]
```

Check if this actuator is "connected" with other actions.

Remarks

It answers for the actions and for nothing else. Tasks are not actions and do not make an actuator connected; ask `has_tasks()` about those. Kept this way deliberately when tasks arrived, because a caller that will never hold a task – an attachment, a poll – reads this and must keep reading the answer it always read.

Returns

true - if the `actuator::actions` list is not empty.
false - if the `actuator::actions` list is empty.

10.1.4.18 `has_tasks()`

```
template<typename action_t>
bool untangle::actuator< action_t >::has_tasks () const [inline]
```

Is there a task waiting to be fired?

The counterpart to `is_connected()`, and the two answer different questions: this one is about `actuator::tasks`, that one about `actuator::actions`. An actuator can hold either, both, or neither.

Remarks

It is what a queue asks to decide whether its worker still has work. Reading `is_connected()` for that would report a batch of tasks as nothing to do, and the work would sit unfired.

A refused task is not a waiting one: `add_task()` answering false means the task was dropped, so this stays false.

Returns

true - a task is waiting; `call_tasks()` has something to fire.
false - no task is waiting. True again after `call_tasks()`, which consumes them.

10.1.4.19 `has_action()`

```
template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]
```

Check if there is certain named action.

Parameters

name	- Name associated with the action.
------	------------------------------------

Returns

true - if name can be found in [actuator::actions_map](#)
false - if name can not be found in [actuator::actions_map](#)

10.1.5 Member Data Documentation

10.1.5.1 tasks

```
template<typename action_t>
tasks_t untangle::actuator< action_t >::tasks
```

Tasks waiting to be fired, in the order [add_task\(\)](#) took them.

Remarks

Held beside [actions](#) and sharing nothing with them: [operator>\(\)](#) fires the actions and leaves these alone, [call_tasks\(\)](#) fires these and leaves the actions alone.

10.1.5.2 owned

```
template<typename action_t>
std::list<action_t> untangle::actuator< action_t >::owned
```

Actions this actuator owns, as added by [add\(action_t&&\)](#).

Remarks

It is a `std::list` because inserting into one never invalidates the address of an element already in it, so every handle already handed out stays valid as further actions are added. A `std::vector` would reallocate and dangle all of them at once.

The documentation for this struct was generated from the following file:

- [actuator.hpp](#)

10.2 untangle::invalid_action Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

Public Member Functions

- [invalid_action](#) (std::string text)
Construct a new invalid action object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

10.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#). When the class object gets invalid, invoking the action will raise an exception to this type.

10.2.2 Constructor & Destructor Documentation

10.2.2.1 invalid_action()

```
untangle::invalid_action::invalid_action (
    std::string text) [inline], [explicit]
```

Construct a new invalid action object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

10.2.3 Member Function Documentation

10.2.3.1 what()

```
const char * untangle::invalid_action::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 11

File Documentation

11.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <cstdint>
00009 #include <exception>
00010 #include <functional>
00011 #include <list>
00012 #include <map>
00013 #include <memory>
00014 #include <string>
00015 #include <tuple>
00016 #include <type_traits>
00017 #include <utility>
00018 #include <vector>
00019
00020 namespace untangle {
00021 // exception
00030 struct invalid_action : std::exception {
00036     explicit invalid_action(std::string text) : message(std::move(text)) {}
00037
00043     const char* what() const noexcept override { return message.c_str(); }
00044
00045 private:
00046     std::string message;
00047 };
00048
00056 template <typename... Args>
00057 auto last_arg(Args&&... args) {
00058     if constexpr (sizeof...(Args) <= 0) {
00059         return 0;
00060     } else {
00061         return std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00062     }
00063 }
00064
00088 template <typename callback_t, typename result_t>
00089 concept task_callback_for =
00090     requires(callback_t& c, result_t& r) { requires std::is_void_v<decltype(c(r))>; } ||
00091     (std::is_void_v<result_t> &&
00092      requires(callback_t& c) { requires std::is_void_v<decltype(c())>; });
00093
00118 using task = std::function<void()>;
00119
00163 template <typename action_t, typename... Args>
00164 task bind_task(action_t action, Args&&... args) {
00165     using result_t = typename action_t::result_type;
00166
00167     static_assert(sizeof...(Args) > 0,
00168         "bind_task: a task must be given a callback as its last argument");
00169
00170     constexpr std::size_t last = sizeof...(Args) - 1;
00171     using callback_t = std::tuple_element_t<last, std::tuple<std::decay_t<Args>...>>;
00172
00173     static_assert(task_callback_for<callback_t, result_t>,
00174         "bind_task: the last argument must be a callback taking the task's result and "
00175         "returning nothing (void()) for a task that returns nothing");
00176 }
```

```

00176
00177 // References into the caller's arguments. What is copied out of it is the task's own, below.
00178 auto pack = std::forward_as_tuple(std::forward<Args>(args)...);
00179
00180 // A task that could not do its job is built empty, and add_task() refuses it on the caller's
00181 // own stack. Only a callable that can be tested is tested: a bare lambda is never empty.
00182 if constexpr (requires { static_cast<bool>(std::get<last>(pack)); }) {
00183     if (!static_cast<bool>(std::get<last>(pack))) {
00184         return {};
00185     }
00186 }
00187 if constexpr (requires { static_cast<bool>(action); }) {
00188     if (!static_cast<bool>(action)) {
00189         return {};
00190     }
00191 }
00192
00193 // The pack is indexed rather than unpacked, because everything but its last element is bound and
00194 // a fold cannot say "all but the last".
00195 return [&<std::size_t... i>(std::index_sequence<i...>) -> task {
00196     return [action = std::move(action), callback = callback_t(std::get<last>(pack)),
00197         ... bound = std::decay_t<decltype(std::get<i>(pack))>(&
00198             std::get<i>(pack))]() mutable -> void {
00199         if constexpr (std::is_void_v<result_t>) {
00200             action(bound...);
00201             callback();
00202         } else {
00203             // One expression, so the result is handed over as an rvalue: a result type that cannot be
00204             // copied is still a result a task can report.
00205             callback(action(bound...));
00206         }
00207     };
00208 } (std::make_index_sequence<last>{}));
00209 }
00210
00262 template <typename action_t>
00263 struct actuator final {
00271     using actions_t = std::list<action_t*>;
00272     using actions_map_t = std::map<std::string, action_t*>;
00286     using tasks_t = std::list<task>;
00287     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
00288         typename action_t::result_type>;
00295     using results_t = std::vector<typename result_t::type>;
00301     using errors_t = std::vector<std::exception_ptr>;
00302
00303     actions_t actions;
00304     actions_map_t actions_map;
00305     results_t results;
00306     errors_t errors;
00307
00314     tasks_t tasks;
00315
00323     std::list<action_t> owned;
00324
00325     actuator() = default;
00331     actuator(const actuator& other) { copy_from(other); }
00332     actuator(actuator&&) noexcept = default;
00333     ~actuator() = default;
00340     actuator& operator=(const actuator& other) {
00341         if (this != &other) {
00342             copy_from(other);
00343         }
00344         return *this;
00345     }
00346     actuator& operator=(actuator&&) noexcept = default;
00347
00355     action_t type() const { return nullptr; }
00356
00364     void reset() {
00365         actions.clear();
00366         actions_map.clear();
00367         results.clear();
00368         errors.clear();
00369         owned.clear();
00370     }
00371
00386     void copy_from(const actuator& other) {
00387         owned = other.owned;
00388
00389         // Old address -> new address, for the owned actions only.
00390         std::map<const action_t*, action_t*> remap;
00391         auto src = other.owned.begin();
00392         auto dst = owned.begin();
00393         for (; src != other.owned.end(); ++src, ++dst) {
00394             remap.emplace(&*src, &*dst);
00395         }
00396         const auto translate = [&remap](action_t* action) {

```

```

00397     const auto it = remap.find(action);
00398     return it != remap.end() ? it->second : action;
00399 };
00400
00401 actions.clear();
00402 for (const auto& action : other.actions) {
00403     actions.push_back(translate(action));
00404 }
00405 actions_map.clear();
00406 for (const auto& entry : other.actions_map) {
00407     actions_map.emplace(entry.first, translate(entry.second));
00408 }
00409 // Named explicitly, like every member here: this function does not default, so anything it
00410 // does not copy is silently lost by every copy of an actuator.
00411 tasks = other.tasks;
00412 results = other.results;
00413 errors = other.errors;
00414 }
00415
00430 void release_owned(const action_t* action) {
00431     for (const auto& a : actions) {
00432         if (a == action) {
00433             return;
00434         }
00435     }
00436     for (const auto& entry : actions_map) {
00437         if (entry.second == action) {
00438             return;
00439         }
00440     }
00441     owned.remove_if([action](const action_t& a) { return &a == action; });
00442 }
00443
00499 template <typename last_t>
00500 void invoke_callback(last_t& last) {
00501     if constexpr (requires { requires std::is_void_v<decltype(last(results.back()))>; }) {
00502         last(results.back());
00503     }
00504 }
00505
00521 template <typename... Args>
00522 void operator()(Args&&... args) {
00523     results.clear();
00524     errors.clear();
00525
00526     // Copied up front, before any action can move the callback out of the argument pack.
00527     auto last = last_arg(args...);
00528
00529     // Dead bindings are collected here and dropped after the loop.
00530     // The actuator does not own the actions it points at, so it must never
00531     // write through action_t* into a std::function belonging to the caller.
00532     std::vector<action_t*> dead_actions;
00533
00534     for (const auto& action : actions) {
00535         // A null pointer or an empty std::function can never be invoked. Calling an
00536         // empty one throws std::bad_function_call, which is not an invalid_action and
00537         // would escape this operator, so drop it instead of invoking it.
00538         if (action == nullptr || !*action) {
00539             dead_actions.push_back(action);
00540             continue;
00541         }
00542
00543         try {
00544             if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00545                 (*action)(std::forward<Args>(args)...);
00546             } else {
00547                 results.push_back((*action)(std::forward<Args>(args)...));
00548                 invoke_callback(last);
00549             }
00550         } catch (const invalid_action&) {
00551             // A dead binding is not coming back: it is recorded and the action is dropped.
00552             errors.push_back(std::current_exception());
00553             dead_actions.push_back(action);
00554         } catch (...) {
00555             // Anything else is the action's own failure. It is recorded, the action is kept, and the
00556             // actions behind it still run.
00557             errors.push_back(std::current_exception());
00558         }
00559     }
00560
00561     for (const auto& dead_action : dead_actions) {
00562         actions.remove(dead_action);
00563         release_owned(dead_action);
00564     }
00565 }
00566
00583 template <typename... Args>

```

```

00584 void invoke_action(const std::string& name, Args&&... args) {
00585     results.clear();
00586     errors.clear();
00587
00588     // Copied up front, before the action can move the callback out of the argument pack.
00589     auto last = last_arg(args...);
00590     const auto it = actions_map.find(name);
00591     if (it == actions_map.end()) {
00592         return;
00593     }
00594
00595     // A null pointer or an empty std::function can never be invoked, and dereferencing a
00596     // null one to try is undefined behaviour before a call is even made. Calling an empty
00597     // one throws std::bad_function_call, which is not an invalid_action and would escape
00598     // this method. Drop it instead, as operator()() does for the actions list.
00599     if (it->second == nullptr || !*it->second) {
00600         const action_t* dead_action = it->second;
00601         actions_map.erase(it);
00602         release_owned(dead_action);
00603         return;
00604     }
00605
00606     try {
00607         if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00608             (*it->second)(std::forward<Args>(args)...);
00609         } else {
00610             results.push_back((*it->second)(std::forward<Args>(args)...));
00611             invoke_callback(last);
00612         }
00613     } catch (const invalid_action&) {
00614         errors.push_back(std::current_exception());
00615         const action_t* dead_action = it->second;
00616         actions_map.erase(name);
00617         release_owned(dead_action);
00618     } catch (...) {
00619         errors.push_back(std::current_exception());
00620     }
00621 }
00622
00657 void call_tasks() {
00658     // Taken by swap, so a callback that adds a task adds it to the *next* pass. Iterating the
00659     // member instead would let a task that re-adds itself keep this loop from ever ending -- and
00660     // it is the same reason a queue built on this takes its batch by move.
00661     tasks_t firing;
00662     firing.swap(tasks);
00663
00664     for (auto& one : firing) {
00665         try {
00666             // The task notifies its own callback: bind_task() sealed the two together.
00667             one();
00668         } catch (...) {
00669             // The task's own failure, or its callback's -- the two are not told apart here, and the
00670             // attention above says why.
00671             errors.push_back(std::current_exception());
00672         }
00673     }
00674 }
00675
00684 void add(action_t* action) { actions.push_back(action); }
00685
00701 action_t* add(action_t&& action) {
00702     owned.push_back(std::move(action));
00703     actions.push_back(&owned.back());
00704     return &owned.back();
00705 }
00706
00713 void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }
00714
00725 action_t* add(const std::string& name, action_t&& action) {
00726     owned.push_back(std::move(action));
00727     const auto [it, inserted] = actions_map.emplace(name, &owned.back());
00728     if (!inserted) {
00729         owned.pop_back();
00730         return nullptr;
00731     }
00732     return &owned.back();
00733 }
00734
00764 bool add_task(task&& one) {
00765     if (!one) {
00766         return false;
00767     }
00768
00769     tasks.push_back(std::move(one));
00770     return true;
00771 }
00772

```

```

00787 void remove(const action_t* action) {
00788     actions.remove_if([&action](const auto& a) { return (action == a); });
00789     release_owned(action);
00790 }
00791
00800 void remove(const std::string& name) {
00801     const auto it = actions_map.find(name);
00802     if (it == actions_map.end()) {
00803         return;
00804     }
00805     const action_t* action = it->second;
00806     actions_map.erase(it);
00807     release_owned(action);
00808 }
00809
00821 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00822
00840 bool has_tasks() const { return !tasks.empty(); }
00841
00849 bool has_action(const std::string& name) const {
00850     return actions_map.find(name) != actions_map.end();
00851 }
00852 };
00853
00871 template <typename action_t, typename arg_t>
00872 void connect_one(actuator<action_t>& target, arg_t&& arg) {
00873     if constexpr (std::is_lvalue_reference_v<arg_t&&> &&
00874         std::is_same_v<std::decay_t<arg_t>, action_t>) {
00875         if (arg != nullptr) {
00876             target.add(&arg);
00877         }
00878     } else {
00879         action_t action(std::forward<arg_t>(arg));
00880         if (action != nullptr) {
00881             target.add(std::move(action));
00882         }
00883     }
00884 }
00885
00928 template <typename action_t, typename... Actions>
00929 actuator<action_t> connect(action_t& A1, Actions&&... An) {
00930     actuator<action_t> target;
00931     connect_one(target, A1);
00932     (connect_one(target, std::forward<Actions>(An)), ...);
00933     return target;
00934 }
00935
00952 template <typename action_t, typename... Actions>
00953 actuator<action_t> connect(std::type_identity_t<action_t>&& A1, Actions&&... An) {
00954     actuator<action_t> target;
00955     connect_one(target, std::move(A1));
00956     (connect_one(target, std::forward<Actions>(An)), ...);
00957     return target;
00958 }
00959
00960 template <typename actuator_t>
00961 void remove_empty_actions(actuator_t& actuator) {
00962     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00963     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
00964         if (it->second == nullptr) {
00965             removable_iterators.push_back(it);
00966         }
00967     }
00968     for (auto it : removable_iterators) {
00969         actuator.actions_map.erase(it);
00970     }
00971 }
00972
00990 template <typename action_t, typename entry_t>
00991 void connect_named_one(actuator<action_t>& target, entry_t&& entry) {
00992     if constexpr (std::is_same_v<std::remove_cvref_t<decltype(entry.second)>, action_t*>) {
00993         if (entry.second != nullptr) {
00994             target.add(entry.first, entry.second);
00995         }
00996     } else {
00997         action_t action(std::forward<entry_t>(entry).second);
00998         if (action != nullptr) {
00999             target.add(entry.first, std::move(action));
01000         }
01001     }
01002 }
01003
01038 template <typename action_t, typename key_t, typename... Actions>
01039 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions&&... An) {
01040     actuator<action_t> target;
01041     connect_named_one(target, std::move(A1));
01042     (connect_named_one(target, std::forward<Actions>(An)), ...);

```

```

01043     return target;
01044 }
01045
01062 template <typename action_t, typename key_t, typename value_t, typename... Actions>
01063     requires(!std::is_same_v<value_t, action_t*>)
01064     actuator<action_t> connect(std::pair<key_t, value_t> A1, Actions&&... An) {
01065     actuator<action_t> target;
01066     connect_named_one(target, std::move(A1));
01067     (connect_named_one(target, std::forward<Actions>(An)), ...);
01068     return target;
01069 }
01070
01071 // generic helpers to remove const qualifier from a function type,
01072 // for instance const member functions
01073 template <typename T>
01074 struct function_remove_const;
01075
01076 template <typename R, typename... Args>
01077 struct function_remove_const<R(Args...)> {
01078     using type = R(Args...);
01079 };
01080
01081 template <typename R, typename... Args>
01082 struct function_remove_const<R(Args...) const> {
01083     using type = R(Args...);
01084 };
01085
01089
01111 template <typename class_t, typename T,
01112     typename action_t = std::function<typename function_remove_const<T>::type>
01113     action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
01114     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
01115         typename action_t::result_type {
01116         // lock() also keeps the object alive for the duration of the call
01117         const auto obj_ = wp.lock();
01118         if (!obj_) {
01119             // inform the actuator about dead binding
01120             throw invalid_action("bind: invalid object");
01121         }
01122         return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
01123     };
01124 }
01125
01146 template <typename class_t, typename T,
01147     typename action_t = std::function<typename function_remove_const<T>::type>
01148     action_t bind(class_t* obj, T class_t::* method) {
01149     return [obj, method](auto&&... args) -> typename action_t::result_type {
01150         if (!obj) {
01151             // inform the actuator about dead binding
01152             throw invalid_action("bind: invalid object");
01153         }
01154         return ((obj)->*method)(std::forward<decltype(args)>(args)...);
01155     };
01156 }
01157
01158 } // namespace untangle

```

Index

actions_t
 untangle::actuator< action_t >, 32
actuator, 1
 untangle::actuator< action_t >, 33
add
 untangle::actuator< action_t >, 38, 39
add_task
 untangle::actuator< action_t >, 40

bind
 namespace untangle: functions, 18, 19
bind_task
 untangle, 23

call_tasks
 untangle::actuator< action_t >, 37
connect
 namespace untangle: functions, 15, 16, 17, 18
connect_named_one
 untangle, 25
connect_one
 untangle, 24
copy_from
 untangle::actuator< action_t >, 34

errors_t
 untangle::actuator< action_t >, 32

has_action
 untangle::actuator< action_t >, 42
has_tasks
 untangle::actuator< action_t >, 42

invalid_action
 untangle::invalid_action, 44
invoke_action
 untangle::actuator< action_t >, 37
invoke_callback
 untangle::actuator< action_t >, 35
is_connected
 untangle::actuator< action_t >, 41

last_arg
 untangle, 23

namespace untangle: functions, 15
 bind, 18, 19
 connect, 15, 16, 17, 18

operator()
 untangle::actuator< action_t >, 36

operator=
 untangle::actuator< action_t >, 33
owned
 untangle::actuator< action_t >, 43

release_owned
 untangle::actuator< action_t >, 34
remove
 untangle::actuator< action_t >, 40, 41
reset
 untangle::actuator< action_t >, 34
results_t
 untangle::actuator< action_t >, 32

task
 untangle, 22
tasks
 untangle::actuator< action_t >, 43
tasks_t
 untangle::actuator< action_t >, 32
type
 untangle::actuator< action_t >, 33

untangle, 21
 bind_task, 23
 connect_named_one, 25
 connect_one, 24
 last_arg, 23
 task, 22
untangle::actuator< action_t >, 29
 actions_t, 32
 actuator, 33
 add, 38, 39
 add_task, 40
 call_tasks, 37
 copy_from, 34
 errors_t, 32
 has_action, 42
 has_tasks, 42
 invoke_action, 37
 invoke_callback, 35
 is_connected, 41
 operator(), 36
 operator=, 33
 owned, 43
 release_owned, 34
 remove, 40, 41
 reset, 34
 results_t, 32
 tasks, 43

- tasks_t, [32](#)
- type, [33](#)
- untangle::invalid_action, [43](#)
 - invalid_action, [44](#)
 - what, [44](#)
- untangle::task_callback_for, [27](#)
- what
 - untangle::invalid_action, [44](#)